

Software Engineering Program

Giancarlo Succi

Structure

- ✧ Frontal lecture mostly on fundamentals
 - ✧ In person on Monday
- ✧ Online practical sessions
 - ✧ Online on Wednesday
- ✧ The specific plan of the lectures will be announced as we progress

Remember the grading strategy

- ✧ Oral
- ✧ Notice that at the first session the oral can be substituted by a written test consisting of 10 questions, with 3 points per questions
- ✧ Very motivated students may ask for an additional project:
 - ✧ That would require an additional effort
 - ✧ Requires to do the exam in the first session
 - ✧ Reduces the questions of the written questions to 3

Students with special needs



Contact the instructor

Program of the course

- ✧ Premises
- ✧ Design of software systems
 - ✧ Introduction to UML
 - ✧ Design Patterns
 - ✧ Architectures
- ✧ Engineering the development of software systems
 - ✧ Focus on the Strategy Pattern
 - ✧ Software Engineering in Python (and a bit of other languages)
 - ✧ Software Engineering for AI systems in Python

Premises

- ✘ Sub-system and module
- ✘ Information hiding
- ✘ Coupling
- ✘ Cohesion
- ✘ Simplicity

Sub-system and module

- Sub-system
 - Performing specific task or subset of responsibilities (procedural approach)
 - Set of classes (OO approach)
 - Example: DBMS and error processing system
- Module
 - More language-specific
 - Set or library of functions performing specific tasks (procedural approach)
 - Classes (OO approach)

Information hiding

- Module must hide its internal implementation
- Module accessed only through public interface
 - No direct access to internal data & private methods
 - Data accessed through a well defined set of accessor methods
- Use abstraction to define modules & interfaces
- Changing implementation (given no change to interface) should have no effect on rest of system



Low coupling

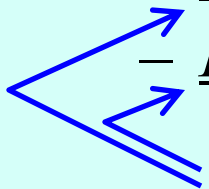


- Two modules are loosely coupled
 - If interconnections and dependencies are weak
 - Satisfying info hiding better than high coupling
- (Increasing) coupling order
 - Methods of a module calling another method's
 - Data coupling/control coupling
 - Class of a module is a subclass of another module's
 - Module(s) making use of specific features of compiler or calls to specific API procedures of the OS
 - I/O coupling
 - Common coupling
 - Content coupling

Avoid!

Cohesion

- Cohesive module: all its elements directed toward performing a single task
- Increasing magnitude of cohesion
 - Coincidental: parts grouped together for no reason
 - Logical: logically related parts, no other interactions
 - Temporal: parts processed within same time limit
 - Procedural: control flows from one part to another
 - Communicational: parts related by same I/O
 - Sequential: output of one is the input of another
 - Informational: access to same data structure
 - Functional: all elements for one single concept



Best to achieve

Simplicity

- Build only needed code; don't try to anticipate future needs
- Refactoring
 - Restructuring a working system to make it simpler
- Simplicity at different levels
 - Method level: short methods with small signatures
 - Module level: small public interface
 - System level:
 - Avoid “middle-man” modules and global variables
 - Minimize info and control paths
 - Keep inheritance hierarchies small
 - At all level: avoid code duplications

